

Embedded systems exp 28

```
#include <systemc.h>

SC_MODULE(counting_semaphore) {

    int sem;

    void task() {
        while(true) {
            wait(1, SC_SEC);
            if(sem > 0) {
                sem--;
                cout << "Task Accessing Resource at "
                    << sc_time_stamp()
                    << " Remaining: " << sem << endl;
                wait(2, SC_SEC);
                sem++;
            }
        }
    }

    SC_CTOR(counting_semaphore) {
        sem = 2;
        SC_THREAD(task);
        SC_THREAD(task);
        SC_THREAD(task);
    }
};

int sc_main(int argc, char* argv[]) {
    counting_semaphore obj("Counting");
    sc_start(10, SC_SEC);
}
```

EDAplayground

NewRunSave*Copy*

ResourcesCommunityHelpPlaygroundsProfile

Brought to you by

DOULOS

DOULOS does not endorse training material from other suppliers on EDA Playground.

Languages & Libraries

Testbench + Design

C++/SystemC

Libraries

NoneSystemC 2.3.3

Tools & Simulators

C++

Compile Options

-DSC_INCLUDE_FX

Select...

☐ Use -pedantic -Wall -Wextra

Run Options

Run Options

☐ Use run.bash shell script

☐ Open EPWave after run

☐ Show output text file

☐ Show output SVG file

☐ Download files after run

testbench.cpp

design.cpp

```
1 #include <systemc.h>
2 SC_MODULE(counting_semaphore) {
3     int sem;
4     void task() {
5         while(true) {
6             wait(1, SC_SEC);
7             if(sem > 0) {
8                 sem--;
9                 cout << "Task Accessing Resource at "
10                  << sc_time_stamp()
11                  << " Remaining: " << sem << endl;
12                 wait(2, SC_SEC);
13                 sem++;
14             }
15         }
16     }
17     SC_CTOR(counting_semaphore) {
```

```
1 // Code your design here.
2 // Uncomment the next line for SystemC modules.
3 // #include <systemc>
```

LogShare

Warning: (W505) object already exists: Counting.task. Latter declaration will be renamed to Counting.task_1
In file: ../../src/sysc/kernel/sc_object_manager.cpp:153
Task Accessing Resource at 1 s Remaining: 1
Task Accessing Resource at 1 s Remaining: 0
Task Accessing Resource at 3 s Remaining: 0
Task Accessing Resource at 4 s Remaining: 0
Task Accessing Resource at 5 s Remaining: 0
Task Accessing Resource at 6 s Remaining: 0
Task Accessing Resource at 7 s Remaining: 0
Task Accessing Resource at 8 s Remaining: 0
Task Accessing Resource at 9 s Remaining: 0
Done